

Causal Masking, in Detail

How a model is kept from reading the answer · video 3:05

This lesson answers one question: during training a GPT sees the whole text at once, so what stops it from reading the next word? The **causal mask**: before the softmax, every score for a future token is set to $-\infty$, so it gets exactly zero attention. The lesson checks it on GPT-2's real weights and trains the same tiny model with and without the mask: without it, the model learns to copy instead of predict.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 7).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. The mask, by hand	▶ 0:08-0:45	2	☐
2. Why $-\infty$ and not 0	▶ 0:45-0:59	2	☐
3. Checked on GPT-2	▶ 0:59-1:23	2	☐
4. Training without the mask	▶ 1:23-2:14	2	☐
5. Efficiency, inference, and the code	▶ 2:14-2:57	3	☐

Before you start: How LLMs Work episodes 5-6 (attention, softmax) and Deep Dive episode 4 (the mask leaks order). Code: `code/d07_causal_mask` (GPT-2 small, about 500 MB, plus two tiny models trained in a few minutes on a CPU).

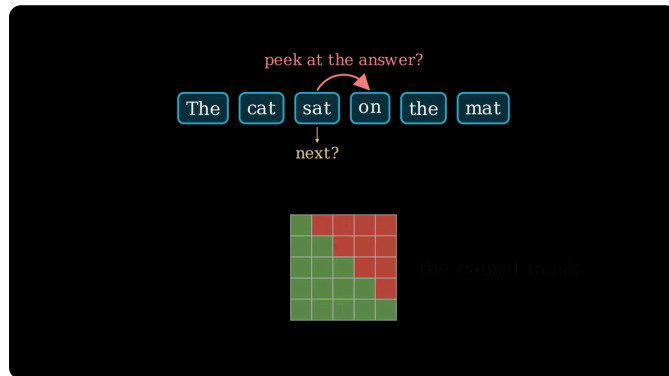
CONCEPT 1 OF 5

The mask, by hand

▶ Video 0:08–0:45

A GPT is trained to predict the next token, but during training it is given the whole text at once, including every answer. What keeps it honest is the **causal mask**.

Take the attention scores for four tokens: each row is a query, each column a key. The cells above the diagonal are the **future**. The mask sets those scores to $-\infty$; then softmax turns each row into weights. Since $e^{-\infty} = 0$, the future gets exactly zero weight, and each row still sums to 1. Row 2, for example, has scores 1.0 and 2.0 for the tokens it may see, giving weights 0.269 and 0.731.



▶ 0:21 During training the whole text is visible: one triangle of numbers keeps each token from seeing the ones after it.



▶ 0:43 Scores above the diagonal become $-\infty$; softmax turns them into exactly 0, and each row still sums to 1.

KEY IDEA

Mask before the softmax, with $-\infty$: the weights are then a proper distribution over the past only.

Check 1: The mask, by hand

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

In a causal mask for 64 tokens, how many of the 64×64 scores are set to $-\infty$?

.....

1.2 CALCULATION

Row 3 has scores 0.5, 1.0, 2.0 for the tokens it may see. What weight goes to token 3?

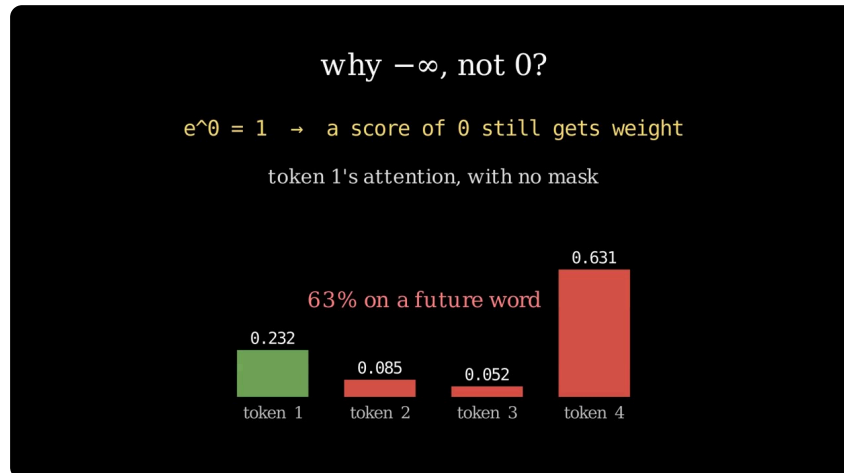
.....

CONCEPT 2 OF 5

Why $-\infty$ and not 0

▶ Video 0:45–0:59

Why minus infinity, and not just zero? A score of zero still counts: $e^0 = 1$. Without any mask, the first token's row (scores 2.0, 1.0, 0.5, 3.0) would put **63%** of its attention (0.631) on token 4, a word that hasn't happened yet.



▶ 0:58 With no mask, token 1 would give 63% of its attention to token 4, a word that hasn't happened yet.

KEY IDEA

“Zero score” is not “no attention”: only $-\infty$ (or a very large negative number) removes a token from the softmax.

Check 2: Why $-\infty$ and not 0

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 HANDS-ON CODE

Try it yourself. Fill the future with 0 instead of $-\infty$. How much attention does token 1 now give to the three future tokens together?

```
masked0 = scores.masked_fill(mask, 0.0)
print(masked0.softmax(dim=-1).round(decimals=3))
```

2.2 TRUE OR FALSE

“Setting future scores to 0 is enough to hide the future.”

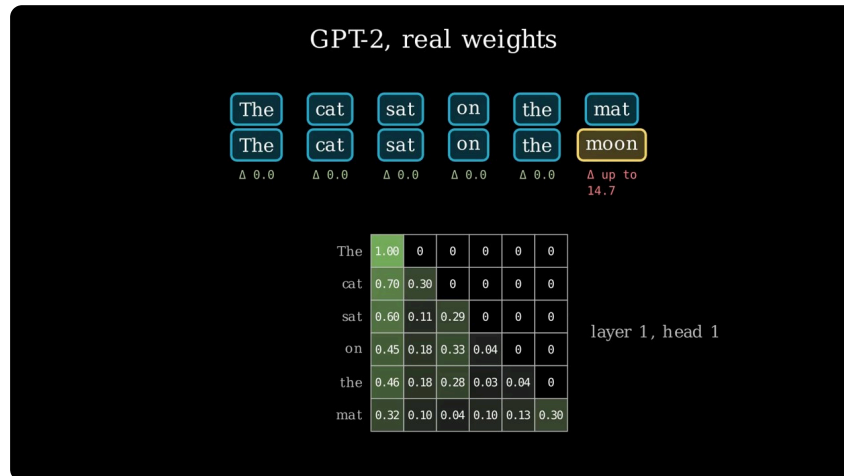
True ☐ False ☐

CONCEPT 3 OF 5

Checked on GPT-2

▶ Video 0:59-1:23

With GPT-2's real weights: *The cat sat on the mat*, then the same sentence ending in *moon*. At every earlier position, the final-layer outputs are identical: a largest difference of exactly **0.0**. Only the last position changes (by up to 14.7). In a real attention pattern from the first layer, everything above the diagonal is zero, and the first token can only attend to itself (weight 1.00).



▶ 1:21 Change “mat” to “moon”: earlier positions differ by exactly 0.0. Layer 1, head 1: zeros above the diagonal.

KEY IDEA

In a causal model the future cannot change the past: each position's output depends only on the tokens up to it.

Check 3: Checked on GPT-2

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 HANDS-ON CODE

Try it yourself. Now change the *second* word: compare “The cat sat on the mat” with “The dog sat on the mat”. Which positions change?

```
b = tok("The dog sat on the mat", return_tensors="pt").input_ids
with torch.no_grad():
    hb = gpt2(b, output_hidden_states=True).hidden_states[-1][0]
print([(ha[i] - hb[i]).abs().max().item() for i in range(6)])
```

3.2 MULTIPLE CHOICE

In a causal model, the first token's attention weights in every head are...

- A. uniform over the sentence
- B. 1.0 on itself
- C. 0 everywhere
- D. 1.0 on the last token

CONCEPT 4 OF 5

Training without the mask

▶ Video 1:23-2:14

The same tiny GPT (818,241 parameters) is trained twice for 1,500 steps on Shakespeare; the only difference is one flag, the mask on or off. Without the mask, the training loss falls to **0.04**: the model simply reads the next character. Tested honestly, given only the past (each prefix fed separately), its loss is **7.36**, worse than guessing uniformly among 65 characters ($\ln 65 = 4.17$). With the mask: **1.52** in training and **1.73** on the honest test.

	mask on	mask off
training loss	1.52	0.04
honest loss (past only)	1.73	7.36
random guessing over 65 characters: $\ln 65 = 4.17$		

▶ 1:57 Without the mask: training loss 0.04, but honest loss 7.36, worse than random guessing (4.17).

mask on	mask off
Capper: MARTIUS: My bethat's one faces. BUCKEND: Lady a Kingless, it. Look, anoal now	learned to copy, not to predict ... 65 empty lines ...

▶ 2:13 The masked model writes Shakespeare-like text; the unmasked one outputs 65 empty lines, then gibberish.

Asked to write, the masked model produces names, line breaks and almost-words. The unmasked one learned to copy, not to predict: with nothing to copy, it outputs 65 empty lines, then gibberish.

KEY IDEA

A training loss is only meaningful if the model sees exactly what it will see when used. Without the mask, the task becomes copying.

Check 4: Training without the mask

Answer all 2 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

What loss does a model get by guessing uniformly among 65 characters?

.....

4.2 SHORT ANSWER

Why is a very low training loss a warning sign here, not a success?

.....

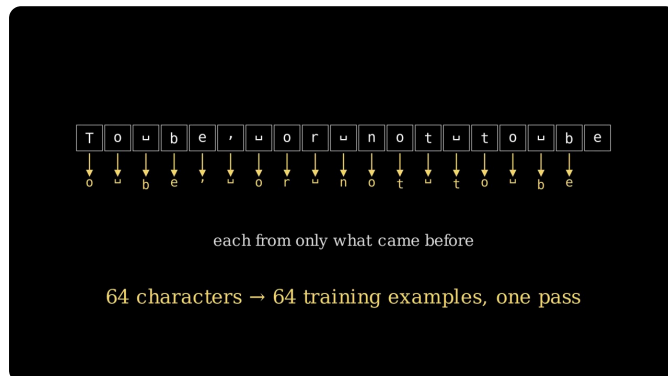
CONCEPT 5 OF 5

Efficiency, inference, and the code

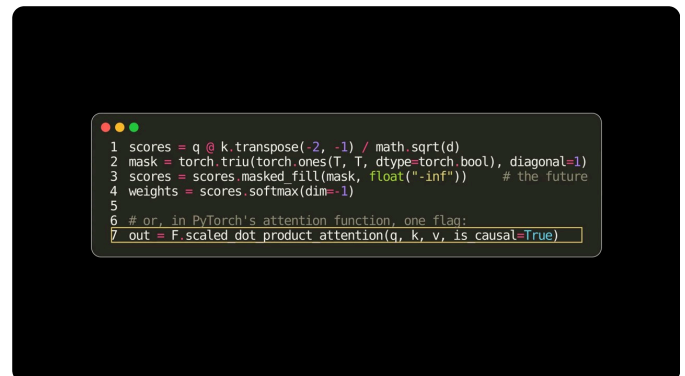
▶ Video 2:14-2:57

The mask also makes training efficient. Each position gets its own prediction, using only what came before, so one 64-character text yields **64 training examples** in a single pass.

At inference, the new token is always the last, so it may see everything before it: with a KV cache, a single query against all cached keys needs no mask at all. Models like **BERT** drop the mask on purpose: they see both sides, but cannot write left to right.



▶ 2:27 Every position predicts the next character from the past only: one text gives as many examples as characters.



▶ 2:56 The mask in one line, or one flag in PyTorch's attention function.

In code, the mask is one line, `masked_fill` of the upper triangle with $-\infty$ before the softmax, or a single flag: `F.scaled_dot_product_attention(q, k, v, is_causal=True)`.

KEY IDEA

The mask lets a model learn every next-token prediction of a text in parallel, while keeping each one honest.

Check 5: Efficiency, inference, and the code

Answer all 3 questions before looking at the answers. Pass mark: all correct.

5.1 CALCULATION

A batch of 32 texts of 64 characters: how many next-character predictions are trained in one step?

5.2 TRUE OR FALSE

“When generating one new token with a KV cache, the causal mask must still be applied.”

True ☐ False ☐

5.3 MULTIPLE CHOICE

Which model is trained without a causal mask?

- A. GPT-2
- B. Qwen2.5
- C. BERT
- D. The episode's masked tiny GPT

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. The mask, by hand · rewatch ▶ 0:08-0:45

1.1 2,016. The strict upper triangle: $64 \times 63 / 2 = 2,016$; the other 2,080 are allowed.

1.2 0.629. $e^2 / (e^{0.5} + e^1 + e^2) = 7.389 / 11.75 \approx 0.629$, as on screen.

2. Why $-\infty$ and not 0 · rewatch ▶ 0:45-0:59

2.1 About 29%: row 1 becomes 0.711, 0.096, 0.096, 0.096. Each zero score still contributes $e^0 = 1$ to the softmax; only the last row, which has no future, is unchanged.

2.2 False. Softmax turns a score of 0 into a positive weight; you need $-\infty$.

3. Checked on GPT-2 · rewatch ▶ 0:59-1:23

3.1 Position 0 (“The”) is unchanged (0.0); positions 1-5 all change (13.1, 1.8, 3.9, 1.9, 5.3). Every later token can see “dog”, so all of them are affected; “The” cannot.

3.2 B. It may only see itself, and softmax over one score gives 1.0.

4. Training without the mask · rewatch ▶ 1:23-2:14

4.1 About 4.17. $-\ln(1/65) = \ln 65 \approx 4.17$; the unmasked model's 7.36 is worse because it is confidently wrong.

4.2 The model could see the answer, so the loss measured copying, not prediction; the honest test shows it cannot predict at all. This is a form of data leakage: the target was part of the input.

5. Efficiency, inference, and the code · rewatch ▶ 2:14-2:57

5.1 2,048. 32×64 ; every position of every text is a training example.

5.2 False. The single new query is the last position; every cached key is in its past, so nothing needs masking.

5.3 C. BERT sees both sides of each token; it fills in blanks rather than writing left to right.